

SQL

(sanjay chauhan)

1. What is SQL?

SQL stands for Structured Query Language. It's used to interact with databases — like to get data, insert new data, update or delete existing data. It's the standard language for working with relational databases like MySQL, PostgreSQL, or SQL Server.

2. What is the primary key?

A primary key is a column (or set of columns) in a table that uniquely identifies each row. It can't have duplicate or NULL values. For example, in a student table, the student ID can be a primary key because each student has a unique ID.

3. What is the foreign key?

A foreign key is a column in one table that links to the primary key of another table. It helps maintain relationships between tables. For example, in an `orders` table, `customer_id` can be a foreign key that refers to the `id` in the `customers` table.

4. What are constraints in sql?

Constraints in SQL are rules applied to columns to control the type of data that can go into them. They help keep data accurate and consistent. Examples include:

PRIMARY KEY – makes sure each row is unique

FOREIGN KEY – maintains relationship between tables

NOT NULL – column can't have NULL values

UNIQUE – all values in the column must be different

CHECK – sets a condition that data must meet

DEFAULT – sets a default value if none is provided

5. What is the difference between delete and truncate?

The main difference is:

DELETE removes rows one by one and can have a **WHERE** condition. It keeps the table structure and logs each row deletion, so it's slower.

TRUNCATE removes all rows at once, without logging each one. It's faster but you can't use **WHERE**.

Also, with **DELETE**, you can roll back (if in a transaction), but **TRUNCATE** can't

be rolled back in some databases.

6. What does join explain its type?

A join is used to combine data from two or more tables based on a related column.

Types of joins:

INNER JOIN – returns only matching rows from both tables.

LEFT JOIN – returns all rows from the left table, and matched rows from the right. If there is no match, the right side will be NULL.

RIGHT JOIN – returns all rows from the right table, and matched rows from the left.

FULL JOIN – returns all rows when there is a match in one of the tables.

CROSS JOIN – returns the cartesian product (all combinations) of two tables.

7. What is group by clause in sql?

The **GROUP BY** clause is used to group rows that have the same values in a column. It's mostly used with aggregate functions like **SUM()**, **COUNT()**, **AVG()**, etc., to perform calculations on each group. For example, if you want total sales by city, you can group by city and sum the sales.

8. What is the difference between where and having clauses?

WHERE is used to filter rows **before** grouping — it works on individual rows.

HAVING is used to filter **after** grouping — it works on grouped data.

Example:

Use **WHERE** to filter sales above 1000,

Use **HAVING** to filter total sales per city above 5000.

9. Explain union and union all

UNION and **UNION ALL** are used to combine results from two **SELECT** queries.

UNION removes duplicate rows from the result.

UNION ALL keeps all rows, including duplicates.

Both require the same number of columns with compatible data types in both queries.

10. What is a subquery in sql?

A subquery is a query inside another query. It's used when you need the result of one query to help in another.

For example, you can use a subquery to find employees who earn more than the average salary. It can be in **SELECT**, **WHERE**, or **FROM** clauses.

11. What is normalization in sql ? explain its types 1NF 2NF 3NF and BCNF?

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity.

Here are the main types:

1NF (First Normal Form):

No repeating groups or arrays. Each column should have atomic (single) values.

2NF (Second Normal Form):

Must be in 1NF and all non-key columns should fully depend on the **whole** primary key (used in tables with composite keys).

3NF (Third Normal Form):

Must be in 2NF and no column should depend on another non-key column (removes transitive dependency).

12. what will be the query to add the new column in a table?

```
ALTER TABLE table_name
```

```
ADD column_name datatype;
```

13. What is the stored procedure in sql?

A stored procedure is a set of SQL statements saved in the database that you can run again and again.

It's like a function — you write it once, and call it whenever needed. It can take input, do some processing (like insert, update, or select), and return output.

Useful for reusability, better performance, and security.

14. write a basic syntax of stored procedure

```
DELIMITER //
```

```
CREATE PROCEDURE GetAllStudents()
```

```
BEGIN
```

```
    SELECT * FROM students;
```

```
END //
```

```
DELIMITER ;
```

To run it, CALL GetAllStudents();

15. What is View in SQL?

A **View** in SQL is a virtual table based on the result of a **SELECT** query. It doesn't store data itself — it just shows data from one or more tables.

It's useful when you want to simplify complex queries or give limited access to certain columns.

```
CREATE VIEW view_name AS
```

```
SELECT column1, column2 FROM table_name;
```

16. What is the difference between a view and a table?

A table stores the data physically, while a **VIEW** is a virtual representation that dynamically pulls data from one or more tables.

17. What is an aggregate function that provides examples?

Aggregate functions perform calculations on a group of rows and return a single value.

Common examples:

COUNT() – counts number of rows

SUM() – adds up values

AVG() – gives average

MAX() – finds highest value

MIN() – finds lowest value

18. Explain the distinct keyword in sql?

The **DISTINCT** keyword is used to remove duplicate values from the result.

It returns only unique rows for the selected columns.

19. What are the ACID properties in SQL?

ACID stands for **Atomicity**, **Consistency**, **Isolation**, and **Durability** — they ensure reliable transactions in a database.

Atomicity – A transaction is all or nothing. If one part fails, the whole transaction fails.

Consistency – Keeps the database in a valid state before and after the transaction.

Isolation – Transactions run independently without interfering with each other.

Durability – Once a transaction is committed, it stays saved even if the system crashes.

20. What is a transaction in SQL?

A transaction in SQL is a set of operations that are executed as a single unit of work. It makes sure that either all the steps are completed successfully or none at all.

It follows the ACID properties to keep data safe and consistent.

21. Explain commit and rollback?

COMMIT is used to save all the changes made in the current transaction permanently in the database.

ROLLBACK is used to undo the changes made in the current transaction if something goes wrong.

BEGIN;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;

-- something goes wrong

ROLLBACK; -- changes undone

-- or if everything is fine

COMMIT; -- changes saved

22. What is a CASE statement in SQL?

The **CASE** statement in SQL is like an **if-else** condition. It lets you apply logic and return values based on conditions.

SELECT

name, salary,

CASE

WHEN salary > 50000 THEN 'High'

WHEN salary > 30000 THEN 'Medium'

ELSE 'Low'

END AS salary_level

FROM employees;

23. What is the COALESCE function in SQL?

The **COALESCE** function returns the **first non-NULL** value from the list you

give.

It's useful when you want to replace NULL with a default value.

```
SELECT COALESCE(address, 'No address') FROM customers;
```

If the address is NULL, it will show 'No address' instead.

24. What is the purpose of the limit clause in sql?

The **LIMIT** clause is used to restrict the number of rows returned by a query.

It's helpful when you only want a few results — like top 5 or top 10 rows.

```
SELECT * FROM employees LIMIT 5;
```

25. What is a composite key?

A **composite key** is a combination of two or more columns used together as a primary key to uniquely identify a row.

It's used when no single column is unique by itself.

Example: In a **course_enrollment** table, **student_id** and **course_id** together can be a composite key, because the same student can take multiple courses, and the same course can have many students.

26. Explain the alter table command?

The **ALTER TABLE** command is used to make changes to an existing table's structure.

Add column, **ALTER TABLE** students **ADD** age **INT**;

Modify column, **ALTER TABLE** students **MODIFY** age **VARCHAR(10)**;

Drop column, **ALTER TABLE** students **DROP COLUMN** age;

27. What is an index?

An **index** in SQL is like a shortcut that helps the database find data faster, just like an index in a book.

28. What is SELF JOIN?

A **SELF JOIN** is a regular join where a table is joined with itself. It is useful when we want to compare the rows within the same table.

Finding Pairs of Products with the Same Price

```
SELECT
```

```
P1.ProductName AS Product1,  
P2.ProductName AS Product2,  
P1.Price  
FROM  
    Products P1  
JOIN  
    Products P2  
ON  
    P1.Price = P2.Price  
AND  
    P1.ProductID < P2.ProductID;
```

29. What are stored functions in SQL?

A **stored function** in SQL is a set of SQL statements that performs a task and returns a single value.

It's similar to a stored procedure, but it **must return a value** and can be used in queries like a built-in function.

```
CREATE FUNCTION get_discount(price DECIMAL)  
RETURNS DECIMAL  
DETERMINISTIC  
BEGIN  
    RETURN price * 0.10;  
END;  
SELECT name, get_discount(price) FROM products;
```

30. What are window functions in SQL, and when should they be used?

Window functions perform calculations across a set of rows **related to the current row**, without collapsing them into a single result like **GROUP BY** does.

They're useful when you want **aggregates** but still keep **row-level detail**.

31. How do you perform a full outer join in databases that don't support it natively?

In databases like MySQL that don't support **FULL OUTER JOIN** directly, you can simulate it by combining a **LEFT JOIN** and a **RIGHT JOIN** with **UNION**.

```
SELECT a.id, a.name, b.value
FROM tableA a
LEFT JOIN tableB b ON a.id = b.id
UNION
SELECT a.id, a.name, b.value
FROM tableA a
RIGHT JOIN tableB b ON a.id = b.id;
```

32. What is the difference between a correlated subquery and a non-correlated subquery?

Correlated subquery: Depends on the outer query. It runs **once for each row** of the outer query.

```
SELECT name
FROM employees e
WHERE salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department = e.department
);
```

Non-correlated subquery: Runs **independently** of the outer query. It executes **once**, and the result is used by the outer query.

```
SELECT name
FROM employees
WHERE salary > (
    SELECT AVG(salary) FROM employees
```


);

33. What is the difference between COUNT(*), COUNT(column_name), and COUNT(DISTINCT column_name)?

COUNT(*): Counts **all rows**, including ones with NULLs.

COUNT(column_name): Counts **non-NULL values** in that column.

COUNT(DISTINCT column_name): Counts **unique non-NULL** values in that column.

34. How would you calculate a running total or cumulative sum in SQL?

To calculate a **running total** (or cumulative sum) in SQL, you can use the **SUM()** function with the **OVER()** clause.

```
SELECT
    order_id,
    customer_id,
    amount,
    SUM(amount) OVER (ORDER BY order_id) AS running_total
FROM orders;
```

35. How do you retrieve the top N records for each group in a table?

To get the **top N records for each group**, use **window functions** like **ROW_NUMBER()** with **PARTITION BY**.

```
SELECT *
FROM (
    SELECT *,
        ROW_NUMBER() OVER (PARTITION BY customer_id
        ORDER BY amount DESC) AS rn
    FROM orders
) AS ranked
WHERE rn <= 2;
```

36. What is the difference between a RANK(), DENSE_RANK(), and ROW_NUMBER() window function?

RANK(): Gives the same rank to ties but skips the next rank(s).

DENSE_RANK(): Gives the same rank to ties without skipping ranks.

ROW_NUMBER(): Gives a unique number to each row, even if values are tied.

37. What is the difference between IN and EXISTS? When would you use one over the other?

IN: Checks if a value exists in a list or result set.

EXISTS: Checks if a subquery returns **any row** (true/false).

When to use:

Use **IN** when the subquery returns **a small, fixed list**.

Use **EXISTS** when checking **row existence**, especially with **correlated subqueries** — it's often faster for large data.

38. What is the difference between CASE and IF statements in SQL?

CASE: Standard SQL, used for conditional logic **within queries** (like **SELECT**).

IF: More procedural, used mainly **inside stored procedures or functions** (in MySQL, etc.).

39. How do you handle data type mismatches in joins?

To handle data type mismatches in joins, **explicitly convert** one column's data type to match the other using **CAST()** or **CONVERT()**.

SELECT *

FROM orders o

JOIN customers c ON o.customer_id = CAST(c.id AS VARCHAR);

